

Continual-Learning GNN Intrusion Detection System

Complete technical documentation
backend, API and user interface

Samarth Divekar

Final-year B.Tech project

github.com/samarthdivekar/Continuous-learning-IDS

Generated 5 October 2026 from the source tree

What this document is

This is a reference for the whole system: every Python module and the functions inside it, every HTTP endpoint, every screen of the console, and every button, dropdown and slider on those screens — with what each one calls on the backend and what comes back.

The reference tables were extracted from the source code itself (Python signatures and docstrings through the `ast` module, interface controls by scanning the dashboard files), so they match the code exactly rather than a description of it. The explanatory text around them was written to match.

Python modules	40 under <code>src/</code> , 20 experiment scripts, 3 operational scripts
Functions and methods	214 functions, 124 class methods
HTTP endpoints	47 routes across three applications
Interface	8 tabs, 54 buttons, 15 dropdowns, 9 other inputs
Tests	74 automated tests, run on every push by GitHub Actions

Contents

1	The system in one page	5	API reference (all 47 routes)
2	How data flows, end to end	6	The user interface
3	Backend reference		6.1 Design system · 6.2 The shell
	3.1 Ingestion · 3.2 Preprocessing · 3.3 Graph building		6.3–6.10 Every tab, control by control
	3.4 Models · 3.5 Training · 3.6 Evaluation	7	Three journeys through the stack
	3.7 Drift · 3.8 Product layer · 3.9 Explanations	8	Measured results
	3.10 Database · 3.11 Utilities · 3.12 Serving	9	Running and operating the system
4	Experiment scripts	10	Glossary

1 · The system in one page

The problem. An intrusion detector is trained once and then meets attacks nobody showed it. Retrain it on a new attack and it forgets the old ones — catastrophic forgetting. Never retrain and it goes stale. Most detectors also judge each connection on its own, so they cannot see patterns that exist only *between* machines.

The approach. Traffic is modelled as a graph: computers are nodes, connections (flows) are edges, and the model classifies the edges. Learning is continual — the model meets one new attack type at a time and keeps the old ones through two mechanisms working together: a penalty protecting the weights that mattered before (EWC) and a small rehearsal buffer of old traffic (replay). A drift detector (ADWIN) decides *when* retraining is needed, instead of retraining on a timer.

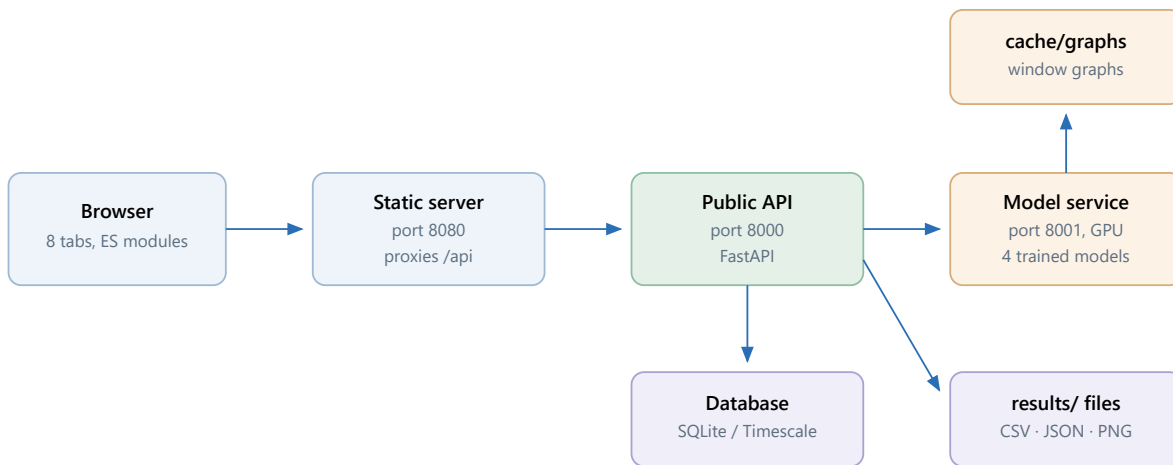
The product on top. A detector that prints 100,000 alerts is unusable, so the system groups alerts into incidents, explains each in plain language, says when it is too unsure to decide, and proposes a containment rule that a human approves or rejects. Nothing is ever executed automatically.

The five layers

Layer	What it does	Where it lives	Technology
Data	reads the public capture files, cleans and samples them, stores flows	<code>src/ingestion</code> , <code>src/preprocessing</code>	pandas, pyarrow
Graph + model	builds windows of 5,000 flows into graphs; classifies every edge	<code>src/graph</code> , <code>src/models</code>	PyTorch, PyTorch Geometric
Learning	continual training, replay buffer, EWC, drift detection, evaluation	<code>src/training</code> , <code>src/evaluation</code> , <code>src/drift</code>	PyTorch, river
Serving	two HTTP services (public API and model service) plus a database	<code>src/api</code> , <code>src/db</code>	FastAPI, SQLAlchemy, SQLite/TimescaleDB
Interface	the eight-tab console an analyst uses	<code>dashboard/</code>	plain ES modules, Chart.js, d3-force

Why two backend services

The heavy model work runs in its own process (`src/api/ml_app.py` , port 8001) and the public API (`src/api/app.py` , port 8000) forwards model calls to it. This keeps model loading out of the request path, and lets the console stay usable when the model service is down: the results tabs read files and keep working, and a banner says which tabs do not.



Request path. The browser talks only to port 8080, which serves the files and proxies `/api` to the public API; model work is forwarded again to the model service. Saved experiment results are read straight from disk, which is why those tabs survive a model outage.

2 · How data flows, end to end

One flow record, from a public dataset to a coloured line on the screen.

#	Stage	What happens	Produced by	Stored as
1	Raw capture	The error-corrected CIC-IDS2017 / CSE-CIC-IDS2018 releases are read; 2018 is streamed straight from its zip and each flow is kept with probability 0.15, decided without looking at the label.	<code>src/ingestion/loader.py</code>	zip in <code>data/raw</code>
2	Cleaning	Column names canonicalised, impossible values dropped, "attempted" attacks relabelled benign, labels mapped onto 8 categories.	<code>src/ingestion/columns.py</code> , <code>labels.py</code>	—
3	Scaling and split	Features log-scaled then standardised using training data only; flows ordered in time and cut into tasks (one new attack type each) with train / validation / test splits.	<code>src/preprocessing/</code>	<code>flows.parquet</code>
4	Window graphs	Every 5,000 consecutive flows become one graph: hosts are nodes, flows are edges carrying ~50 features. Cached under a hash of the settings that produced them.	<code>src/graph/window_builder.py</code>	<code>cache/graphs/</code>
5	Training	Models learn the tasks in order. Ours keeps a replay buffer of old subgraphs and an EWC penalty; a checkpoint is written after every task.	<code>src/training/learners.py</code>	<code>cache/checkpoints/</code>
6	Evaluation	After each task the model is scored on every task seen so far: accuracy, macro-F1, retention, false-positive rate, forgetting.	<code>src/evaluation/</code>	<code>results/<dataset>/<mode>/</code>
7	Serving	The model service loads the final checkpoints; the public API reads result files and forwards model calls.	<code>src/api/</code>	HTTP JSON
8	Display	Each tab fetches what it needs and renders charts, tables or a force-directed graph.	<code>dashboard/js/views/</code>	the screen

The rule the whole project follows: every number shown anywhere is read from a file an experiment script wrote. Nothing in the interface is typed in by hand, and when an experiment has not been run the panel says "not run yet" rather than showing a number.

What a window graph contains

- **Nodes** — the distinct hosts in those 5,000 flows. A node's only feature is its degree (how many connections it has). This is deliberate: the model never receives IP addresses, so it cannot memorise them.
- **Edges** — one per flow, carrying the scaled features (duration, packet counts, byte rates, TCP flags, ...) and the training label.

- **Direction** — preserved. Incoming and outgoing neighbours are aggregated separately, which is how the model distinguishes a scanner (many outgoing) from the victim of a flood (many incoming).

3 · Backend reference

Every module, class and function in `src/`, grouped by package. Signatures and one-line purposes come from the code itself.

3.1 · Ingestion `src/ingestion`

Reads the public datasets: the two file layouts, the canonical column names, and the mapping from dozens of raw label strings onto the eight categories used throughout.

`src/ingestion/columns.py` · 98 lines

Column-name normalisation across CIC-IDS2017 / CSE-CIC-IDS2018 releases.

Definition	Purpose
<code>_basic_snake(name)</code>	—
<code>canonical_name(name)</code>	—
<code>normalise_columns(columns)</code>	Return {original: canonical}. Raises if two columns collapse to one name.

`src/ingestion/labels.py` · 53 lines

Raw label -> attack category mapping for both datasets.

Definition	Purpose
<code>is_attempted(raw_label)</code>	—
<code>base_label(raw_label)</code>	—
<code>to_category(raw_label)</code>	—
<code>build_label_table(raw_labels, attempted_policy)</code>	Map every distinct raw label to its final category (None = drop the flow).

`src/ingestion/loader.py` · 134 lines

CSV flow-record loader.

Definition	Purpose
class ZipMember	A CSV inside a zip archive, read as a stream (the 2018 release is 36 GB)
<code>open()</code>	—
<code>ensure_extracted(raw_dir, dataset_info)</code>	—
<code>list_csvs(source)</code>	—
<code>_open(src)</code>	—
<code>_read_one_csv(path, attempted_policy, flow_sample, rng, chunksize)</code>	—
<code>load_flows(csv_files, attempted_policy, flow_sample_fraction, seed)</code>	—

3.2 · Preprocessing `src/preprocessing`

Turns raw flows into model-ready arrays: scaling fitted on training data only, the task sequence (one new attack type at a time) and the train/validation/test split.

`src/preprocessing/clean.py` · 51 lines

Cleaning: infinities, NaNs, duplicates, chronological ordering.

Definition	Purpose
<code>feature_columns(df)</code>	—
<code>clean_flows(df)</code>	Return a cleaned, chronologically ordered copy plus a stats dict.

`src/preprocessing/pipeline.py` · 168 lines

End-to-end data preparation: raw CSV -> processed flow table -> cached graphs.

Definition	Purpose
class ProcessedData	—
<code>df()</code>	—
<code>feature_columns()</code>	—
<code>n_tasks()</code>	—
<code>task_categories()</code>	—
<code>tabular(task, split)</code>	—
<code>graphs(task, split)</code>	—
<code>processed_dir(cfg)</code>	—
<code>prepare_dataset(cfg, force, csv_files)</code>	—
<code>build_graph_cache(cfg, df, feats, force)</code>	—
<code>load_processed(cfg)</code>	—

`src/preprocessing/scaling.py` · 65 lines

Feature scaling: signed log1p -> StandardScaler (fit on task 1 train only) -> clip.

Definition	Purpose
class FeatureScaler	—
<code>_log(x)</code>	—
<code>fit(X, columns)</code>	—
<code>transform(X, chunk)</code>	—
<code>inverse_transform(Z)</code>	Approximate original units (exact unless the value was clipped). Used to
<code>save(path)</code>	—
<code>load(cls, path)</code>	—

`src/preprocessing/tasks.py` · 96 lines

Chronological partition of the flow stream into one task per attack category.

Definition	Purpose
class Segment	—
<code>find_segments(df, gap_seconds, min_segment_flows)</code>	—
<code>assign_tasks(df, segments)</code>	Add `segment_id` and `task_id` columns. `df` must be sorted by ts.
<code>segments_table(segments)</code>	—

3.3 · Graph building `src/graph`

Builds and caches window graphs, samples subgraphs for training, and implements the IP-remap tests that check the model is not memorising host identities.

`src/graph/ip_remap.py` · 76 lines

IP-remapping evaluation modes (brief §7: IP leakage).

Definition	Purpose
<code>_copy_meta(src, dst)</code>	—
<code>permute_ips_in_graph(g, seed)</code>	Random bijection on node ids (equivalent to permuting IP addresses).
<code>reassign_sources(g, pool_size, seed, node_feature_kind)</code>	Replace each flow's source host by an independent random host from a pool.
<code>make_transform(mode, seed, pool_size)</code>	—

`src/graph/sampling.py` · 103 lines

Edge-centric neighbour sampling (GraphSAGE-style) for large windows.

Definition	Purpose
class IncidenceIndex	CSR-style index: node -> incident edge ids (both directions).
<code>sample(frontier, fanout, rng)</code>	—
<code>sample_subgraph(g, target_edges, fanouts, rng, index)</code>	—
<code>iter_training_subgraphs(g, cfg_ns, rng, shuffle)</code>	Yield (sub)graphs for training on window `g`.

`src/graph/window_builder.py` · 277 lines

Windowed graph construction: flows -> one PyG `Data` graph per time window.

Definition	Purpose
<code>_window_local_ids(ts, window_cfg)</code>	—
<code>temporal_split(ordinal, win_task, tcfg)</code>	Per-task chronological split. Returns split codes indexed like `ordinal`; gap windows get -1.
<code>attack_aware_temporal_split(df, ordinal, win_task, task_categories, tcfg)</code>	Temporal split over each attack's own active span.
<code>assign_windows_and_splits(df, window_cfg, split_cfg, window_fraction, task_categories)</code>	Add `window_id` (global, chronological) and `split` columns.

Definition	Purpose
<code>node_features(num_nodes, edge_index, kind)</code>	Node features carry no identity. `constant` = E-GraphSAGE default.
<code>build_window_graph(src_ip, dst_ip, edge_attr, y, node_feature_kind, **meta)</code>	—
<code>edge_labels(g, label_mode)</code>	Graphs store the category id in `y`; binary labels are derived on demand
<code>graphs_from_table(df, feature_cols, node_feature_kind)</code>	Build graphs for every window in `df`, in chronological window order.
<code>cache_key(cfg)</code>	—
<code>graph_cache_dir(cfg, cache_root)</code>	—
<code>save_graphs(graphs, path)</code>	—
<code>load_graphs(path)</code>	—

3.4 · Models `src/models`

The networks and the two memory mechanisms: the edge-featured GraphSAGE model, the per-flow baseline, the EWC penalty and the replay buffer.

`src/models/baseline_xgb.py` · 47 lines

Static tabular baseline: XGBoost trained once on task 1, then frozen.

Definition	Purpose
<code>class StaticXGBoost</code>	—
<code>fit(X, y, sample_weight)</code>	—
<code>predict_proba(X)</code>	Probabilities over the GLOBAL class space (unseen classes get 0).
<code>predict(X)</code>	—

`src/models/egraphsage.py` · 99 lines

E-GraphSAGE: edge-featured, inductive GraphSAGE for flow (edge) classification.

Definition	Purpose
<code>class EdgeSAGEConv</code>	—
<code>forward(x, edge_index, edge_attr)</code>	—
<code>message(x_j, edge_attr)</code>	—
<code>class EGraphSAGELayer</code>	—
<code>forward(x, edge_index, edge_attr)</code>	—
<code>class EGraphSAGE</code>	—
<code>node_embeddings(x, edge_index, edge_attr)</code>	—
<code>edge_features(x, edge_index, edge_attr, target_edges)</code>	—
<code>forward(x, edge_index, edge_attr, target_edges)</code>	Return logits for `target_edges` (indices into edge_index), or all edges.

Definition	Purpose
<code>forward_with_embedding(x, edge_index, edge_attr)</code>	(logits, embedding): embedding = the classifier's hidden layer, one vector per

`src/models/ewc.py` · 173 lines

Elastic Weight Consolidation (Kirkpatrick et al., 2017), online variant.

Definition	Purpose
class EWC	—
<code>active()</code>	True only once at least one task has been consolidated (TRAP 1).
<code>penalty(model)</code>	—
<code>estimate_fisher(batches, loss_fn, max_batches)</code>	Diagonal empirical Fisher: $E_batches[(\partial L_task / \partial \theta)^2]$.
<code>consolidate(batches, loss_fn, max_batches, lr, tag)</code>	Call AFTER a task / adaptation cycle has finished training.
<code>stability_ratio(lr)</code>	—
<code>state_dict()</code>	—
<code>load_state_dict(state, device)</code>	—

`src/models/ffnn.py` · 28 lines

Plain feed-forward network on per-flow features (ablation / fallback).

Definition	Purpose
class FFNN	—
<code>forward(x)</code>	—
<code>forward_with_embedding(x)</code>	(logits, embedding): embedding = the last hidden layer (after ReLU).

`src/models/replay_buffer.py` · 214 lines

Experience replay buffers.

Definition	Purpose
class GraphReplayBuffer	—
<code>categories()</code>	—
<code>add(graph)</code>	Offer one window graph to every eligible category pool. Returns the
<code>add_many(graphs)</code>	—
<code>sample(n)</code>	Class-balanced, fixed-budget sample of stored windows (see module doc).
<code>snapshot()</code>	Cheap copy of the buffer's state (graphs are shared by reference, never mutated).
<code>restore(snap)</code>	—
<code>summary()</code>	—
class TabularReplayBuffer	Per-category reservoir of individual flow rows for the FFNN ablation.
<code>categories()</code>	—
<code>add(X, y_cat)</code>	Vectorised reservoir update, category by category. Benign rows are

Definition	Purpose
<code>sample(n)</code>	—
<code>snapshot()</code>	—
<code>restore(snap)</code>	—
<code>summary()</code>	—

3.5 · Training `src/training`

One uniform interface over every model, so the harness can treat a gradient-boosted tree and a graph network identically.

`src/training/learners.py` · 479 lines

Continual learners: a uniform interface over every model in the comparison.

Definition	Purpose
class LearnStats	—
class BaseLearner	—
<code>learn(graphs, tag, epochs)</code>	—
<code>predict_proba(g)</code>	—
<code>adapts()</code>	—
class XGBLearner	—
<code>learn(graphs, tag, epochs)</code>	—
<code>predict_proba(g)</code>	—
<code>adapts()</code>	—
class _TorchLearner	Shared optimiser / EWC / replay plumbing for GNN and FFNN learners.
<code>snapshot_state()</code>	Everything <code>learn()</code> mutates: weights, optimiser moments, EWC anchors/Fisher,
<code>restore_state(st)</code>	—
<code>_post_init(model)</code>	—
<code>_make_optimizer()</code>	—
<code>_step(loss)</code>	—
<code>_check_finite(loss, tag)</code>	—
class GNNLearner	—
<code>_labels(g)</code>	—
<code>_augment(g)</code>	Improvement 7 — topology augmentation. With probability p the window's
<code>_loss_on(sub, weight, category_balanced)</code>	—
<code>_raw_task_loss(model, g)</code>	Plain CE on one window — used for Fisher estimation (EWC TRAP 2: no penalty, no replay).
<code>learn(graphs, tag, epochs)</code>	—

Definition	Purpose
<code>predict_proba(g)</code>	—
<code>predict_details(g)</code>	(logits, embeddings) per flow — for novelty scoring and embedding drift.
class FFNNLearner	—
<code>_to_labels(y_cat)</code>	—
<code>_raw_task_loss(model, batch)</code>	—
<code>learn(graphs, tag, epochs)</code>	—
<code>predict_proba(g)</code>	—
<code>predict_details(g)</code>	(logits, embeddings) per flow — for novelty scoring and embedding drift.
<code>class_weights(labels, num_classes, power, clip)</code>	$w_c = (N / (K_{\text{present}} * n_c))^{**} \text{power}$ for classes present; 1.0 otherwise.
<code>load_learner_checkpoint(learner, path)</code>	Restore model weights (and EWC state) from a <code>run_continual</code> checkpoint.
<code>make_learner(name, cfg, n_features, num_classes, device, node_in)</code>	<code>`cfg["model_overrides"][name]`</code> (list of key=value) is applied for this model
<code>snapshot(learner)</code>	Deep copy (used by the drift experiment to branch a learner).

3.6 · Evaluation `src/evaluation`

The task-sequence harness, the streaming simulation, the metrics, and the trust features (open-set novelty detection and conformal abstention).

`src/evaluation/calibration.py` · 47 lines

Calibration of predicted probabilities: does a 0.9 confidence mean 90 % correct?

Definition	Purpose
<code>reliability(probs, y, n_bins)</code>	—
<code>calibration_errors(probs, y, n_bins)</code>	—

`src/evaluation/conformal.py` · 56 lines

Calibrated abstention with class-conditional (Mondrian) split conformal prediction.

Definition	Purpose
<code>fit_class_thresholds(probs, y, alpha, min_n)</code>	—
<code>prediction_sets(probs, q, fallback)</code>	Boolean (n, K) membership. Classes without their own threshold use <code>`fallback`</code> .
<code>evaluate(probs, y, sets, benign)</code>	—

`src/evaluation/continual.py` · 179 lines

Task-sequence evaluation harness (the core comparison table).

Definition	Purpose
<code>predict_graphs(learner, graphs, label_mode)</code>	—
<code>_concat(preds)</code>	—
<code>save_checkpoint(learner, path)</code>	—
<code>run_task_sequence(cfg, data, model_names, out_dir, eval_split, eval_transforms, checkpoint_dir, tasks)</code>	Run all models through the task sequence; write CSVs into `out_dir`.
<code>_merge_write(path, new)</code>	Replace rows of the models in `new`, keep rows of every other model already
<code>_write(out_dir, long_rows, summary_rows)</code>	—

`src/evaluation/metrics.py` · 78 lines

Metric definitions (brief §6). All metrics are computed from raw prediction

Definition	Purpose
<code>binary_view(y, label_mode)</code>	—
<code>core_metrics(y_true, y_pred, label_mode)</code>	—
<code>category_recall(y_cat, y_pred, category_id, label_mode)</code>	—
<code>confusion(y_true, y_pred, num_classes)</code>	—
<code>forgetting_metrics(R)</code>	$R[i, j]$: performance on task j after training task i (NaN where undefined).

`src/evaluation/open_set.py` · 101 lines

Open-set novelty detection: flag flows that belong to NO known class, before

Definition	Purpose
<code>_normalize(z)</code>	—
<code>fit_prototypes(z, y, known, max_per_class, seed)</code>	—
<code>novelty_scores(logits, z, known, prototypes)</code>	—
<code>calibrate(val_known_scores, target_fpr)</code>	Threshold so that `target_fpr` of known validation flows would be flagged.
<code>detection_metrics(known, novel, threshold)</code>	—
<code>propose_clusters(z_flagged, cat_flagged, k_range, max_points, seed)</code>	Cluster flagged flows; report how pure the proposed 'new category' is.

`src/evaluation/stream.py` · 326 lines

Drift-driven streaming simulation (brief §5.5, Phase 7).

Definition	Purpose
class StreamRunner	—
<code>stream_graphs()</code>	—
<code>_mixed_pairs(per_task)</code>	Pair consecutive stream tasks and interleave each pair's windows evenly: a window at

Definition	Purpose
<code>evaluate(stream_index, seen_tasks, window_id, ts)</code>	—
<code>uses_adwin()</code>	—
<code>gated()</code>	—
<code>_select_labels(graphs)</code>	Improvement 1 — active learning. With <code>`drift.label_budget` = N > 0</code> , an
<code>_gate_metrics()</code>	Held-out check used by the gate: VALIDATION windows of every task seen so far.
<code>adapt(graphs, tag)</code>	—
<code>run(max_windows)</code>	—
<code>start(pretrained)</code>	<code>pretrained=True</code> : the learner was already restored from a task-0
<code>finish(last, n)</code>	—
<code>_flow_signal(g, probs)</code>	—
<code>step(k, g)</code>	Process stream window number <code>`k`</code> : predict, log, monitor, maybe adapt.
<code>manual_adapt(tag)</code>	Adaptation cycle requested from outside (POST /retrain in the demo).
<code>frames()</code>	—
<code>select_label_rows(margins, budget, strategy, rng)</code>	Rows (flows) an analyst is asked to label, given top-1 minus top-2 margins.

3.7 • Drift detection `src/drift`

The ADWIN monitor that decides when traffic has changed enough to justify retraining.

`src/drift/adwin_monitor.py` • 139 lines

ADWIN drift monitor (Bifet & Gavaldà 2007) via ``river``.

Definition	Purpose
class DriftEvent	—
class ADWINMonitor	—
<code>estimate()</code>	—
<code>calibrate(values)</code>	Feed baseline values without raising events.
<code>_feed(values)</code>	Update value by value; return (level_before, level_after) for every detection.
<code>_event(detections, stream_index, window_id, ts)</code>	—
<code>update(value, stream_index, window_id, ts)</code>	One value = one window (granularity 'window').
<code>update_window(values, stream_index, window_id, ts)</code>	All per-flow values of one window (granularity 'flow'). At most one event per window.
<code>notify_adapted()</code>	Called after any adaptation (ADWIN-, schedule- or manually triggered).

Definition	Purpose
<code>events_as_dicts()</code>	—
<code>river_available()</code>	(usable, reason). Lets callers report a missing detector instead of crashing.

3.8 · Product layer `src/product`

What turns a classifier into something an analyst can use: incident grouping, proposed containment actions, and export for security information and event management (SIEM) tools.

`src/product/incidents.py` · 97 lines

Incident-level alerting with a false-alarm budget (improvement 3).

Definition	Purpose
<code>class_DSU</code>	—
<code>find(a)</code>	—
<code>union(a, b)</code>	—
<code>calibrate_threshold(probs, y, budget, benign)</code>	Smallest τ such that $\leq$ `budget` of validation benign flows get attack confidence $\geq \tau$.
<code>build_incidents(src, dst, probs, class_names, threshold, ts, y_cat, benign, min_flows)</code>	—
<code>incident_metrics(incidents, y_cat, benign)</code>	Precision at incident level (majority of flows truly malicious) and how much

`src/product/response.py` · 50 lines

Response actions with human approval (improvement 15) — DRY RUN ONLY.

Definition	Purpose
<code>_valid_ip(s)</code>	—
<code>propose_action(incident)</code>	Return {action, target, rationale, rules{windows, linux}} for one incident.

`src/product/siem.py` · 62 lines

SIEM export (ArcSight CEF) for incidents.

Definition	Purpose
<code>_sev(incident)</code>	Incident severity ($\log_{10}(\text{flows}) \times \text{mean confidence}$) -> CEF 0-10.
<code>_ts(value)</code>	—
<code>incident_to_cef(incident, window_id, model, host)</code>	—
<code>incidents_to_cef(incidents, window_id, model, host)</code>	—

3.9 · Explanations `src/explain`

Why one flow was flagged: feature attribution, how much evidence came from neighbouring flows, and the structural facts, summarised into a sentence by rules (no language model involved).

`src/explain/explain.py` · 116 lines

Per-alert explanations (improvement 2).

Definition	Purpose
<code>pretty(name)</code>	—
<code>explain_edge(learner, g, edge, feature_names, scaler, top_k)</code>	—
<code>summarize(exp, class_names)</code>	—

3.10 · Database `src/db`

The schema shared by SQLite (default) and PostgreSQL/TimescaleDB (via DATABASE_URL), plus seeding.

`src/db/models.py` · 154 lines

Database schema (brief §8) as SQLAlchemy ORM models.

Definition	Purpose
<code>class Base</code>	—
<code>class FlowRecord</code>	—
<code>class GraphWindow</code>	—
<code>class ReplayBufferEntry</code>	—
<code>class Prediction</code>	—
<code>class Metric</code>	—
<code>class DriftEventRow</code>	—
<code>class WindowStat</code>	—
<code>class ResponseAction</code>	Improvement 15: a proposed containment action awaiting (or after) human decision.
<code>utcnow()</code>	—

`src/db/seed.py` · 124 lines

Load committed results + window metadata into the database.

Definition	Purpose
<code>_f(v)</code>	—
<code>_ts(v)</code>	—
<code>seed(dataset, label_mode, Session)</code>	—
<code>main()</code>	—

`src/db/session.py` · 90 lines

Engine/session management.

Definition	Purpose
<code>database_url()</code>	—
<code>make_engine(url)</code>	—
<code>init_db(engine)</code>	—
<code>get_sessionmaker(url)</code>	—

Definition	Purpose
<code>reset_for_tests(url)</code>	—

3.11 · Utilities `src/utils`

Configuration loading and overrides, tuned-setting selection, and safe checkpoint loading.

`src/utils/config.py` · 68 lines

Configuration loading: default.yaml <- dataset overlay <- CLI overrides.

Definition	Purpose
<code>deep_merge(base, overlay)</code>	—
<code>_parse_value(raw)</code>	—
<code>apply_overrides(cfg, overrides)</code>	Apply `a.b.c=value` style overrides.
<code>load_config(dataset, overrides, config_path)</code>	—
<code>resolve_path(cfg, key)</code>	—
<code>num_classes(cfg)</code>	—
<code>class_names(cfg)</code>	—

`src/utils/logging.py` · 13 lines

Definition	Purpose
<code>get_logger(name)</code>	—

`src/utils/repro.py` · 65 lines

Seeding, device selection and run metadata (recorded next to every result).

Definition	Purpose
<code>set_seed(seed)</code>	—
<code>get_device(pref)</code>	—
<code>_pkg_version(name)</code>	—
<code>run_info(cfg, extra)</code>	—
<code>write_run_info(path, cfg, extra)</code>	—

`src/utils/safe_load.py` · 48 lines

Loading model checkpoints without handing an attacker a shell.

Definition	Purpose
<code>_is_local_artifact(path)</code>	—
<code>load_checkpoint(path, map_location)</code>	Checkpoint contents, read with weights_only=True whenever possible.

`src/utils/selection.py` · 79 lines

Validation-selected hyper-parameters, shared by experiments and the API service.

Definition	Purpose
<code>_selection_root(cfg, dev)</code>	—

Definition	Purpose
<code>tuning_overrides(cfg, dev)</code>	—
<code>selected_ewc_overrides(cfg, dev, cli_keys)</code>	{"source": path None, "overrides": {model_name: [k=v, ...]}}, exact model names only.
<code>apply_selection(cfg, dev, cli_overrides)</code>	Apply tuning selected.yaml, re-apply CLI overrides on top, attach per-model EWC selections.

3.12 · Serving `src/api`

The two FastAPI applications, the service layer that owns the models, the live-stream runner, the read-only results API and the printable report.

`src/api/app.py` · 511 lines

Public REST API (brief §8, layer 4).

Definition	Purpose
class FlowIn	—
class IngestIn	—
class PredictIn	—
class ProposeIn	—
class DecisionIn	—
class DemoStartIn	—
class State	—
<code>_remote(method, path, timeout, **kw)</code>	—
<code>series_key(model, policy)</code>	Dashboard series id. Each model's DEFAULT deployment keeps the bare model name
<code>_action_dict(r)</code>	—
<code>store_predictions(Session, flow_ids, result)</code>	—
<code>create_app(database_url, service, load_models)</code>	—

`src/api/ml_app.py` · 140 lines

ML service container (brief §8, layer 3).

Definition	Purpose
class PredictBody	—
class DemoBody	—
<code>lifespan(app)</code>	—
<code>health()</code>	—
<code>_drift_status()</code>	ADWIN needs river, whose compiled extension some machines block: say so instead of failing.
<code>_gpu_name()</code>	—
<code>predict(body)</code>	—

Definition	Purpose
<code>scan_incidents(limit, model, threshold)</code>	—
<code>incidents(window_id, model, threshold)</code>	—
<code>incident_report(window_id, incident_id, model, threshold)</code>	—
<code>explain(window_id, edge, model)</code>	—
<code>windows_catalog()</code>	—
<code>retrain()</code>	—
<code>graph(window_id, max_nodes, model)</code>	—
<code>demo_start(body)</code>	—
<code>demo_stop()</code>	—
<code>demo_status()</code>	—

`src/api/report_html.py` · 107 lines

Printable one-page incident report (browser -> Print -> Save as PDF).

Definition	Purpose
<code>fmt_value(value)</code>	Readable feature values: 56737.0 -> "56,737", 0.0 -> "0", 8923.44 -> "8,923.4".
<code>_row(label, value)</code>	—
<code>incident_report_html(rep)</code>	—

`src/api/results.py` · 238 lines

Read-only access to the committed experiment outputs in results/ for the dashboard.

Definition	Purpose
<code>_check(dataset, mode)</code>	—
<code>_records(df)</code>	JSON-safe records (NaN/inf -> None).
<code>_read_csv(path)</code>	—
<code>_read_json(path)</code>	—
<code>_seeds(base, name)</code>	The multi-seed summary written by <code>experiments.aggregate_seeds</code> (<name>_seeds/summary.csv), or None
<code>index()</code>	Which experiments exist on disk, per dataset and label mode.
<code>continual(dataset, mode)</code>	—
<code>confusion(dataset, mode, model, after_task, seed)</code>	—
<code>drift(dataset, mode)</code>	—
<code>loao(dataset, mode)</code>	—
<code>ip_remap(dataset, mode)</code>	—
<code>tuning(dataset)</code>	—
<code>open_set(dataset)</code>	Improvement 6: novelty-detection scores on the next (unseen) task, plus proposed-category clusters.

Definition	Purpose
<code>conformal(dataset)</code>	Improvement 7: class-conditional conformal abstention.
<code>incidents(dataset)</code>	Improvement 3: alert -> incident compression under a false-alarm budget.
<code>adaptation(dataset)</code>	Improvements 4 + 8: gated adaptation and label-budgeted (active-learning) streams, next to the baseline.
<code>run_info(dataset, mode, experiment)</code>	—
<code>data_summary(dataset)</code>	Task segmentation and per-split counts from the processed-data metadata.

`src/api/service.py` · 585 lines

ML service layer: model loading, prediction, graph summaries and the live demo.

Definition	Purpose
class MissingFeaturesError	User-supplied flows lack features the models were trained on. A verdict computed with those
class MLService	—
<code>data_available()</code>	—
<code>data()</code>	—
<code>scaler()</code>	—
<code>window_index()</code>	—
<code>get_window(window_id)</code>	—
<code>checkpoint_dir()</code>	—
<code>load_models()</code>	—
<code>invalidate_predictions()</code>	Call after any change to a model's weights.
<code>_probs(g, model)</code>	Class probabilities for one window, cached per (window, model, model version).
<code>warm_start(learner, task)</code>	Restore `learner` to its state right after training task `task` from the
<code>active_learners()</code>	Live demo learners take precedence once they are READY (warm-started or
<code>flows_to_graph(flows)</code>	Graph of user-supplied flows. Every feature the models were trained on must be present in
<code>predict(flows, window_id, models, max_flows_returned)</code>	—
<code>graph_summary(window_id, max_nodes, model)</code>	Node/edge summary of a window. With `model`, every flow is also
<code>_ips()</code>	—
<code>incidents(window_id, model, threshold)</code>	Improvement 3: flagged flows of a window grouped into incidents (with IPs).
<code>scan_incidents(limit, model, threshold, split, max_incidents)</code>	The queue across many windows, not just one: score the most recent `limit` windows of
<code>incident_report(window_id, incident_id, model, threshold)</code>	Everything needed for a hand-off report: the incident, the proposed action and

Definition	Purpose
<code>explain(window_id, edge, model)</code>	Improvement 2: why was this flow classified the way it was?
<code>list_windows()</code>	Every cached window with its task, split and attack composition (for the explorer).
<code>start_demo(delay_seconds, eval_every, max_windows)</code>	—
<code>stop_demo()</code>	—
<code>request_retrain()</code>	—
class DemoRunner	Replays the chronological stream through several models, writing every
<code>_db(rows)</code>	—
<code>_on_window(row)</code>	—
<code>_on_drift(ev)</code>	—
<code>_on_eval(row)</code>	—
<code>run()</code>	—
<code>describe()</code>	—
<code>_utc(ts)</code>	—
<code>service_config()</code>	—
<code>_nan(v)</code>	—

4 • Experiment scripts

Each script writes files under `results/`; nothing else in the project writes a reported number.

`reproduce_all.py` runs the whole chain in order.

`experiments/aggregate_continual.py` • 65 lines

Cross-seed aggregation of a `run_continual` results folder (pure pandas, no torch).

Definition	Purpose
<code>aggregate(root, seeds)</code>	—
<code>main()</code>	—

`experiments/aggregate_seeds.py` • 112 lines

Combine single-seed result folders into one multi-seed table.

Definition	Purpose
<code>folder_seed(path)</code>	—
<code>load(kind, path, label, seeds)</code>	—
<code>summarise(per_seed, keys, metrics)</code>	—
<code>main()</code>	—

`experiments/benchmark_serving.py` • 155 lines

Serving performance envelope: how fast can the deployed system score traffic?

Definition	Purpose
<code>make_service(device)</code>	A fresh service on `device`. Imported lazily so the device is set before anything loads.
<code>time_windows(fn, windows, repeats)</code>	—
<code>summarise(device, op, cache, lat, flows)</code>	—
<code>run_device(device, n_windows)</code>	—
<code>main()</code>	—

`experiments/common.py` • 77 lines

Shared CLI plumbing for experiment scripts.

Definition	Purpose
<code>base_parser(description)</code>	—
<code>cli_overrides(args)</code>	—
<code>config_from_args(args)</code>	default.yaml < dataset overlay < CLI. No validation selections applied
<code>apply_tuning(cfg, args)</code>	Apply tuning/selected.yaml only (used by the λ sweep, which produces the EWC selection).
<code>apply_selection(cfg, args)</code>	Tuning selected.yaml + per-model EWC λ/γ from the validation sweep.

Definition	Purpose
<code>results_dir(cfg, args, *parts)</code>	—
<code>data_config(cfg)</code>	`cfg` with the dataset's reference seed, for preparing and loading data.

`experiments/make_plots.py` · 184 lines

Regenerate every figure in results/ from the CSVs (never from memory).

Definition	Purpose
<code>_style(model)</code>	—
<code>plot_continual(cdir, models)</code>	—
<code>plot_drift(ddir)</code>	—
<code>plot_loao(ldir)</code>	—
<code>main()</code>	—

`experiments/make_report.py` · 205 lines

Build results/RESULTS.md from the result CSVs (no number is typed by hand).

Definition	Purpose
<code>fmt(m, s, pct, digits)</code>	—
<code>md_table(df)</code>	—
<code>ewc_settings(ds, mode)</code>	Validation-selected λ/γ per model (fallback for runs made before the
<code>continual_section(ds, mode, folder, split)</code>	—
<code>drift_section(ds, mode)</code>	—
<code>loao_section(ds, mode)</code>	—
<code>ipremap_section(ds, mode)</code>	—
<code>tuning_section(ds)</code>	—
<code>main()</code>	—

`experiments/prepare_data.py` · 18 lines

Phase 1 + 4: raw CSVs -> cleaned, task-partitioned, scaled flow table -> cached window graphs.

Definition	Purpose
<code>main()</code>	—

`experiments/reproduce_all.py` · 75 lines

One command that regenerates every reported number and figure from raw data.

Definition	Purpose
<code>run(cmd)</code>	—
<code>main()</code>	—

`experiments/run_calibration.py` · 97 lines

Calibration of the final continual models, beside the conformal results.

Definition	Purpose
<code>plot(rel, summary, path)</code>	—
<code>main()</code>	—

`experiments/run_conformal.py` · 77 lines

Calibrated abstention (improvement 4) on the final continual models.

Definition	Purpose
<code>gather(learner, graphs)</code>	—
<code>main()</code>	—

`experiments/run_continual.py` · 108 lines

Core comparison: every model through the identical chronological task sequence.

Definition	Purpose
<code>is_nonstandard(cfg, overrides)</code>	True when a run departs from the reference protocol: any <code>--set</code> override (per-seed training)
<code>main()</code>	—

`experiments/run_drift_stream.py` · 83 lines

Phase 7: ADWIN-triggered adaptation on a chronological stream.

Definition	Purpose
<code>main()</code>	—

`experiments/run_incidents.py` · 81 lines

Incident grouping + false-alarm budget (improvement 3) on the final models.

Definition	Purpose
<code>main()</code>	—

`experiments/run_ip_remap.py` · 40 lines

Phase 8b: IP-leakage check (brief §7).

Definition	Purpose
<code>main()</code>	—

`experiments/run_loao.py` · 91 lines

Phase 8a: leave-one-attack-out generalisation.

Definition	Purpose
<code>drop_category_edges(g, cat_id)</code>	—
<code>main()</code>	—

`experiments/run_open_set.py` · 119 lines

Open-set novelty detection on the task sequence (improvement 6).

Definition	Purpose
<code>collect(learner, graphs, max_windows, rng)</code>	—
<code>main()</code>	—

`experiments/split_report.py` · 47 lines

What each split strategy leaves to test on, per task.

Definition	Purpose
<code>main()</code>	—

`experiments/stats.py` · 132 lines

Paired comparisons across seeds for the claims the project makes.

Definition	Purpose
<code>final_per_seed(dataset, mode, folder)</code>	Last-task metrics per (model, seed), normal IP mode only.
<code>bootstrap_ci(diffs, n_boot, seed)</code>	—
<code>compare(per_seed, other, metric, sign)</code>	—
<code>main()</code>	—

`experiments/sweep_ewc_lambda.py` · 82 lines

EWC λ / γ sweep (brief §5.3 trap 3), scored on the VALIDATION split.

Definition	Purpose
<code>main()</code>	—

`experiments/tune_val.py` · 66 lines

Small, symmetric hyper-parameter check on the VALIDATION split.

Definition	Purpose
<code>main()</code>	—

5 · API reference

All 47 routes. The public API serves everything twice — at the root for direct use and under `/api` for the dashboard — so a path listed as `/predict` also answers at `/api/predict`.

5.1 Public API `src/api/app.py · port 8000`

	Path	Handler	What it does
GET	<code>/health</code>	<code>health()</code>	—
POST	<code>/ingest</code>	<code>ingest(body)</code>	—
POST	<code>/predict</code>	<code>predict(body)</code>	—
GET	<code>/metrics</code>	<code>metrics(source, run_id, model)</code>	—
GET	<code>/drift-status</code>	<code>drift_status(run_id, limit)</code>	—
GET	<code>/stream/windows</code>	<code>stream_windows(run_id, since_index, limit)</code>	—
POST	<code>/retrain</code>	<code>retrain()</code>	—
GET	<code>/graph/{window_id}</code>	<code>graph(window_id, max_nodes, model)</code>	—
GET	<code>/windows</code>	<code>windows(split, limit)</code>	—
GET	<code>/incidents/scan</code>	<code>scan_incidents(limit, model, threshold)</code>	The queue across the most recent windows, ranked by severity (a shift's worth of work).
GET	<code>/incidents/{window_id}</code>	<code>incidents(window_id, model, threshold)</code>	Improvement 3: the window's flagged flows grouped into incidents, each with a proposed action.
GET	<code>/incidents/{window_id}/report</code>	<code>incident_report(window_id, incident_id, model, threshold, fmt)</code>	A printable hand-off report for one incident (browser Print -> Save as PDF).
GET	<code>/incidents/{window_id}/cef</code>	<code>incidents_cef(window_id, model, threshold)</code>	The window's incidents as ArcSight CEF lines, for a SIEM to ingest. Nothing is sent anywhere.
GET	<code>/explain/{window_id}/{edge}</code>	<code>explain(window_id, edge, model)</code>	Improvement 2: feature, neighbourhood and structural evidence for one flow's verdict.
POST	<code>/actions</code>	<code>propose(body)</code>	Improvement 15: record the proposed containment action for an incident (dry run).
POST	<code>/actions/{action_id}/decision</code>	<code>decide(action_id, body)</code>	An analyst approves or rejects. Nothing is ever executed: approval is recorded as a dry run.

	Path	Handler	What it does
GET	/actions	actions(status, limit)	—
GET	/windows/catalog	windows_catalog()	Every cached window with task, split and attack composition (Graph Explorer).
POST	/demo/start	demo_start(body)	—
POST	/demo/stop	demo_stop()	—
GET	/demo/status	demo_status()	—
GET	/	root()	—

5.2 Results API `src/api/results.py` · read-only

These never touch the model: they read the committed experiment outputs. A missing experiment returns 404 so the interface can say "not run yet".

	Path	Handler	What it does
GET	/results/index	index()	Which experiments exist on disk, per dataset and label mode.
GET	/results/continual	continual(dataset, mode)	—
GET	/results/confusion	confusion(dataset, mode, model, after_task, seed)	—
GET	/results/drift	drift(dataset, mode)	—
GET	/results/loao	loao(dataset, mode)	—
GET	/results/ip_remap	ip_remap(dataset, mode)	—
GET	/results/tuning	tuning(dataset)	—
GET	/results/open_set	open_set(dataset)	Improvement 6: novelty-detection scores on the next (unseen) task, plus proposed-category clusters.
GET	/results/conformal	conformal(dataset)	Improvement 7: class-conditional conformal abstention.
GET	/results/incidents	incidents(dataset)	Improvement 3: alert -> incident compression under a false-alarm budget.
GET	/results/adaptation	adaptation(dataset)	Improvements 4 + 8: gated adaptation and label-budgeted (active-learning) streams, next to the baseline.
GET	/results/run_info	run_info(dataset, mode, experiment)	—
GET	/results/data_summary	data_summary(dataset)	Task segmentation and per-split counts from the processed-data metadata.

5.3 Model service `src/api/ml_app.py` · port 8001, internal

Not exposed to the browser; the public API forwards to it over a pooled connection.

	Path	Handler	What it does
GET	/health	health()	—
POST	/predict	predict(body)	—
GET	/incidents/scan	scan_incidents(limit, model, threshold)	—
GET	/incidents/{window_id}	incidents(window_id, model, threshold)	—
GET	/incidents/{window_id}/report	incident_report(window_id, incident_id, model, threshold)	—
GET	/explain/{window_id}/{edge}	explain(window_id, edge, model)	—
GET	/windows/catalog	windows_catalog()	—
POST	/retrain	retrain()	—
GET	/graph/{window_id}	graph(window_id, max_nodes, model)	—
POST	/demo/start	demo_start(body)	—
POST	/demo/stop	demo_stop()	—
GET	/demo/status	demo_status()	—

5.4 Conventions

- **Errors** — 404 for a window, model or experiment that does not exist; 422 for a value outside its allowed range; 409 when an action was already decided or the incident changed since it was displayed; 503 when the model service is unreachable or its libraries cannot load.
- **Authentication** — optional. Set `GNNIDS_API_KEY` and every route except `/health` requires an `X-API-Key` header.
- **Compression** — responses above 1 KB are gzipped; the largest (the 2018 drift stream) drops from 2,271 KB to 99 KB.
- **Caching** — model scores are cached per window, model and model version, so a repeat request for the same window is roughly four times faster.
- **Safety** — proposed containment actions are recorded, never executed, whether approved or not.

6 • The user interface

The console is deliberately plain: hand-written ES modules, no framework and no build step. Two libraries are loaded from a CDN — Chart.js for the charts and d3-force for the network layout. Every file is served as-is, so what is in the repository is what runs in the browser.

6.1 Design system

Dark by default, with a light theme switched from the sidebar. Colours are CSS custom properties defined once on `:root`. The chrome is quiet (cool greys and one indigo accent for "you are here" and primary actions); colour with meaning is reserved for data. A model keeps the same colour on every chart of every page, so a line can be recognised without reading the legend. Text contrast is measured, not estimated: 16.3:1, 10.0:1 and 5.8:1 for the three text levels in dark, and 4.95:1 for white on a primary button.

Colour roles

Token	Value	Used for
<code>--accent</code>	<code>#6b7bff</code>	chrome accent: the current page, switches, focus
<code>--good</code>	<code>#3ec46d</code>	healthy status, approved decisions
<code>--warn</code>	<code>#fab219</code>	warnings and degraded-service banners
<code>--critical</code>	<code>#ef5f5f</code>	errors, rejected decisions, attack edges
<code>--m-ours</code>	<code>#3987e5</code>	GNN + EWC + replay (our model)
<code>--m-gnn-naive</code>	<code>#d95926</code>	GNN naive retraining
<code>--m-xgb</code>	<code>#199e70</code>	XGBoost static baseline
<code>--m-ffnn</code>	<code>#c98500</code>	FFNN + EWC + replay
<code>--m-ffnn-naive</code>	<code>#d55181</code>	FFNN naive retraining
<code>--m-gnn-ewc</code>	<code>#1f9e1f</code>	GNN with EWC only
<code>--m-gnn-replay</code>	<code>#9085e9</code>	GNN with replay only
<code>--m-joint</code>	<code>#e66767</code>	joint training (upper bound)

Components

Component	Class	What it is
Card	<code>.card</code>	the panel every chart and table sits in; rounded, bordered, with an optional header row
KPI tile	<code>.kpi</code>	one large number with a label above and a detail line below
Segmented control	<code>.seg</code>	a row of buttons where exactly one is active (dataset, label mode, scope)
Toggle switch	<code>.toggle</code>	an on/off switch with a sliding knob (Compare models)
Sub-page tabs	<code>.sub-nav</code>	underlined tabs that split one page into sections (Models, Adaptation & trust)
Status row	<code>.status-row</code>	a coloured dot, a name and a one-word state for API, model service, database and stream
Tag	<code>.tag</code>	small rounded label: severity, 'dry run', a window number
Data table	<code>.data</code>	compact table; numeric columns right-aligned, a highlighted row for the row that matters

Component	Class	What it is
Heat cell	<code>.heat</code>	table cell whose background opacity encodes a value from 0 to 1
Chart	<code>.chart</code>	fixed-height canvas rendered by Chart.js (line, bar, horizontal bar, logarithmic)
Graph stage	<code>.graph-stage</code>	the force-directed network canvas with hover tooltip
Banner	<code>.banner</code>	message under the top bar when something is wrong, naming what still works
Drawer	<code>.drawer</code>	the Help panel sliding in from the right with a glossary
Tour card	<code>.tour</code>	the five-step guided walk, bottom right
Toast	<code>.toast</code>	transient confirmation, bottom right, disappears after a few seconds
Empty state	<code>.empty</code>	dashed box reading 'not run yet' or the error returned

Layout and type

- **Grid** — panels are laid out with CSS grid helpers (`.g2` , `.g3` , `.g4` , `.g-5-7` , `.g-7-5` , `.g-8-4`). Below 1480 px the top bar drops its field captions; below 1200 px the sidebar becomes an icon rail; below 860 px it becomes a drawer opened from a menu button and every grid becomes one column.
- **Type** — Inter for text and JetBrains Mono for anything an analyst might copy (IP addresses, firewall rules, file paths), both self-hosted.
- **Numbers** — shown the way a person reads them: thousands separators, percentages with sensible precision, and never scientific notation (a port renders as 56,737, not 5.674e+04).

6.2 The shell: sidebar, top bar, and the parts that are always there

One HTML page (`dashboard/index.html`) holds a sidebar, a slim top bar and eight empty page sections. `js/main.js` routes between them: changing the URL hash loads that page's module on first use, so nothing is fetched for a page nobody opens.

Sidebar (navigation, status, utilities)

Part	Widget	What it does
Operate / Evaluate	navigation list with icons	one item per page; the current page is highlighted, a LIVE badge marks a running stream, and hovering shows the page's number key
API · Model service · Database · Stream	status rows	polled every 4 seconds from /health and /demo/status; a coloured dot and one word (online, offline, idle, or stream progress)
►	icon button	starts the five-step guided tour
?	icon button	opens the Help drawer (glossary, where to start); opens itself on a first visit
Presentation	icon button	larger type and thicker chart lines for a projector; fine print is hidden
Theme	icon button	toggles dark and light; charts are re-rendered with the new palette

Top bar (context shared by every page)

Control	Widget	What it does
Breadcrumb	text	the current group and page
CIC-IDS2017 / CSE-CIC-IDS2018	segmented control	switches dataset; every open page re-reads its data
Multiclass / Binary	segmented control	named attack types or attack-vs-benign; Binary is disabled for CSE-CIC-IDS2018, whose binary task sequence was withdrawn
Compare models	toggle switch	off shows the deployed model and two reference points; on shows every baseline and ablation
Jump to... (Ctrl K)	button	opens the command palette
≡	icon button	on a phone: opens the sidebar as a drawer

Pages

Group	Page	Module	What it answers
Operate	Overview	<code>views/overview.js</code>	The headline results in one screen
Operate	Incident queue	<code>views/soc.js</code>	Alerts grouped into incidents, with explanations and approve/reject
Operate	Live stream	<code>views/live.js</code>	Replay traffic and watch the model detect drift and adapt
Operate	Graph explorer	<code>views/explorer.js</code>	See one traffic window as a graph of hosts and flows
Operate	Classify	<code>views/classify.js</code>	Classify a held-out window or an uploaded CSV of flows
Evaluate	Models	<code>views/models.js</code>	Accuracy, forgetting, unseen attacks and the IP-leakage test
Evaluate	Adaptation & trust	<code>views/adapt.js</code>	Drift detection, retraining, novelty, abstention and alert load
Evaluate	Reproducibility	<code>views/repro.js</code>	Data, settings and how to reproduce every number

Keyboard

- `1 - 8` — switch page

- `?` — open or close the Help drawer · `Esc` — close it

Always-on behaviour

- **Health polling** — every 4 seconds; the answer is cached server-side for 3 seconds so a dead model service cannot slow the page down.
- **Banner** — appears when the model service or the API is unreachable, naming the tabs that still work. Dismissing it hides that message only; a different problem shows again.
- **API key** — if the server requires one, the console asks for it once and remembers it in the browser. Downloads fetch with the key as a header, so it never appears in a URL.
- **Preferences** — theme, dataset, label mode, compare toggle, chosen model and sub-tab are remembered per browser.

6.3 · Overview `dashboard/js/views/overview.js`

The answer to 'does it work?' in one screen.

- **Plain-words summary** — a sentence built from the result files: how many attack types were learned, how much of the first one is still detected, and how many false alarms per 100,000 flows.
- **Four KPI tiles** — macro-F1 with its standard deviation over seeds, retention of the first attack (with the naive model's figure for contrast), false-positive rate, and drift-triggered retrains.
- **Verdict table** — per model: does it adapt, does it remember, macro-F1, false-positive rate. A tick appears above 80 %, which is stated on the panel rather than left implicit.
- **Attack timeline** — a stacked horizontal bar chart of attack and benign flows per task.
- **Architecture diagram** — inline SVG of the five layers.
- **Dataset panel** — flow and window counts, features, task segmentation, read from the processed metadata (`/results/data_summary`).

6.4 · Incident queue `dashboard/js/views/soc.js`

What an analyst actually works through: alerts grouped into incidents, each explained, each with a proposed containment action awaiting a human decision.

- **Scope** — one window, or a scan of the last 10/20/50 windows ranked by severity (a shift's queue).
- **KPI row** — flows analysed, flagged flows, incidents, and either the share of incidents that are real (checked against ground truth, demo only) or the busiest window in the scan.
- **Incident list** — severity-ranked buttons with category colour, key host, size, confidence and a severity bar; filters narrow by category and severity.
- **Detail panel** — the plain-English explanation, a horizontal bar chart of feature attribution (red pushes towards the verdict, blue against), network context facts, the proposed action with the exact firewall rule for Linux and Windows, and the approve / reject controls.
- **Decision log** — every proposal and decision, filterable, with who decided and when.

Every control on this tab

Control	Widget	Element id	What happens when you use it
One window	button	—	scope switch: a single window
Recent	button	—	
Load incidents	button	soc-go	calls <code>/incidents/{window}</code> or <code>/incidents/scan</code> and renders the queue
↓ CSV	button	soc-csv	downloads the visible incidents as CSV, built in the browser
↓ CEF	button	soc-cef	downloads the window's incidents as CEF lines for a SIEM (<code>/incidents/{window}/cef</code>)
All	button	—	decision log filter: everything
Pending	button	—	decision log filter: awaiting a decision
Approved	button	—	decision log filter: approved
Rejected	button	—	decision log filter: rejected
(one per row)	button	one per row	each incident in the list is a button; clicking it opens that incident in the detail panel
Linux (iptables)	button	—	shows the iptables form of the proposed rule
Windows Firewall	button	—	shows the PowerShell form of the same rule
Copy rule	button	soc-copy	copies the firewall rule shown above to the clipboard
Open report	button	soc-report	opens the printable one-page report (<code>/incidents/{window}/report</code>)
Approve a	button	soc-approve	records the proposed action as approved — a dry run; nothing is executed
Reject r	button	soc-reject	records it as rejected
—	select	soc-win	chooses the traffic window (test windows that contain attacks, biggest first)
—	select	soc-limit	how many recent windows a scan covers

Control	Widget	Element id	What happens when you use it
—	select	<code>soc-model</code>	which detector scores the window (the graph model, the per-flow model, or naive retraining)
—	select	<code>soc-cat</code>	filters the list to one attack category
—	select	<code>soc-sev</code>	filters by severity (critical only, or high and above)
—	input[range]	<code>soc-th</code>	minimum confidence; alerts below it are dropped before grouping
your name	input[text]	<code>soc-analyst</code>	the name stored with the decision
note (optional)	input[text]	<code>soc-note</code>	an optional note stored with the decision

6.5 · Live stream `dashboard/js/views/live.js`

Replays the traffic stream through four models at once and shows drift detection and self-retraining as they happen.

- **Progress and KPIs** — position in the stream, and per-window counts of benign, known-attack and novel/drifted predictions.
- **Live charts** — error rate over the stream with vertical markers where each model adapted.
- **Event feed** — one line per drift event: the error before and after, and whether it triggered retraining or was inside the refractory period.
- **Everything is written to the database** as it happens (windows, metrics, drift events), which is the same path the offline experiment uses.

Every control on this tab

Control	Widget	Element id	What happens when you use it
▶ Start stream	button	<code>lv-start</code>	starts the replay (<code>POST /demo/start</code>) with the chosen speed
⌛ Retrain now	button	<code>lv-retrain</code>	forces an adaptation now (<code>POST /retrain</code>), the operator override
■ Stop	button	<code>lv-stop</code>	stops the replay (<code>POST /demo/stop</code>)
—	input[range]	<code>lv-speed</code>	delay between windows, from fast to slow enough to narrate

6.6 · Graph explorer `dashboard/js/views/explorer.js`

One window as a picture: hosts as dots, flows as lines, laid out by d3-force on a canvas.

- **Why it exists** — attacks have shapes. A port scan fans out from one host; a flood fans in to one victim. That shape is exactly what the graph model can use and a per-flow model cannot.
- **Model-error overlay** — switches the colouring from ground truth to the chosen model's mistakes, so false alarms and missed attacks are visible as edges.
- **Interaction** — drag to pan, wheel to zoom, hover a node for its degree and in/out counts.

Every control on this tab

Control	Widget	Element id	What happens when you use it
Ground truth	button	—	colour edges by their true category
Model errors	button	—	colour edges by whether the model got them right
↗ Fit	button	<code>ex-fit</code>	
(one per row)	button	—	
—	select	<code>ex-model</code>	which model's errors to overlay
—	select	<code>ex-task</code>	which task (attack period) to pick windows from
—	select	<code>ex-split</code>	train, validation or test windows
—	input[range]	<code>ex-nodes</code>	maximum hosts drawn, to keep the layout readable
—	input[checkbox]	<code>ex-attack</code>	show only edges that are attacks

6.7 · Classify `dashboard/js/views/classify.js`

Run the models on demand: either a held-out window with ground truth, or your own flows.

- **Window mode** — pick a test window; every loaded model classifies it and the counts are compared with the truth.
- **Your own flows** — upload a CSV with every CICFlowMeter feature the models were trained on. A file missing any feature in any flow is rejected with HTTP 422 naming the missing features; nothing is imputed, because a verdict computed on zero-filled features would look authoritative and mean nothing. There is no free-form paste box.
- **Output** — a logarithmic bar chart of predicted counts per category per model, a summary table, and for small inputs a per-flow verdict table with confidences.

Every control on this tab

Control	Widget	Element id	What happens when you use it
Classify window	button	<code>cl-run-w</code>	classifies that window with every loaded model (<code>POST /predict</code>)
Classify file	button	<code>cl-run-f</code>	classifies the uploaded file (parsed in the browser, then <code>POST /predict</code>); disabled until a file with address columns is loaded
—	select	<code>cl-win</code>	chooses a held-out window to classify
—	input[file]	<code>cl-file</code>	loads a CSV file and previews its rows and columns (5 MB limit, stated in the interface)

6.8 · Models `dashboard/js/views/models.js (compare.js + general.js)`

The evidence about model quality, in two sub-sections: *Accuracy & forgetting* and *Unseen attacks & IP leakage*.

- **Metric over tasks** — any metric plotted as the task sequence progresses, with a shaded band of ± 1 standard deviation across seeds.
- **Per-category recall matrix** — a heat table: rows are 'after task i', columns are categories, so forgetting is visible as a column going cold.
- **Forgetting (BWT)** — a bar chart; negative means the model lost ground on earlier tasks.
- **Confusion matrix** — for any model after any task.
- **Unseen attacks** — leave-one-attack-out detection per held-out category.
- **IP leakage** — the same trained model scored normally, with host identities permuted, and with sources randomised.

Every control on this tab

Control	Widget	Element id	What happens when you use it
—	select	<code>cmp-metric</code>	which metric the line chart shows
—	select	<code>cmp-rm</code>	which model's recall matrix is displayed
—	select	<code>cmp-cm-model</code>	which model's confusion matrix is displayed
—	select	<code>cmp-cm-task</code>	after which task the confusion matrix is taken
Binary	button	—	scores the unseen-attack test in attack-vs-benign mode
Multiclass	button	—	scores it with named categories

6.9 · Adaptation & trust `dashboard/js/views/adapt.js (drift.js + trust.js)`

When the system noticed traffic had changed and what it did, plus how far its decisions can be trusted. Two sub-sections: *Drift & retraining* and *Trust*.

- **Policy comparison** — ADWIN against a fixed schedule, an oracle that knows the true boundaries, and never adapting: retrain cost against final quality.
- **Error timeline** — per-window error with drift flags and adaptations marked.
- **Novelty** — how well each unseen attack is separated from known traffic, by three scores, plus the clustering that proposes a candidate new category.
- **Abstention** — false alarms before and after the model is allowed to say 'not sure'.
- **Alert load** — how many incidents an analyst faces at each false-alarm budget.
- **Safe adaptation** — all labels against the recommended 100-label hybrid setting, plus the safety gate (single runs, labelled as indicative). The failed budget variants are one sentence of prose.

Every control on this tab

Control	Widget	Element id	What happens when you use it
—	select	<code>dr-run</code>	which stream run to display
—	select	<code>tr-method</code>	which novelty score to chart (energy, max softmax, prototype distance)

6.10 · Reproducibility `dashboard/js/views/repro.js`

How every number was produced, so a reader can check rather than trust.

- **EWC λ sweep** — validation score against the penalty strength, on a logarithmic axis, with the selected value marked.
- **Tuning table** — what was chosen on the validation split, never the test split.
- **Stability ratios** — how sensitive the result is to the setting.
- **Run metadata** — git commit, seeds, device, package versions and timing for each experiment.
- **The command** that regenerates everything.

Every control on this tab

Control	Widget	Element id	What happens when you use it
Copy command	button	<code>rp-copy</code>	

7 · Three journeys through the stack

What actually happens between a click and a pixel.

7.1 Pressing "Load incidents"

1. `soc.js` reads the window, model and confidence slider, then calls `get("/incidents/363?model=gnn_ewc_replay&threshold=0")`.
2. The static server on 8080 proxies `/api/...` to the public API on 8000, passing the API key header through if there is one.
3. The public API forwards to the model service on 8001 over a pooled connection.
4. `MLService.incidents()` loads the cached window graph, scores it (reusing the cached probabilities if that window and model were scored before), and reads the source/destination addresses.
5. `build_incidents()` keeps flows the model called an attack, then unions hosts that share a connection into components, one group per predicted category. Each group gets its key host, size, mean confidence and a severity of $\log(\text{size}) \times \text{confidence}$.
6. `propose_action()` chooses block / rate-limit / isolate from the shape of the incident and writes the exact firewall rule.
7. The browser renders the KPI tiles, the ranked list and the detail panel, then asks `/explain/363/<edge>` for the evidence behind the first incident and draws the attribution chart.

Measured: about 0.24 s per call once the window has been scored once; 3.5 s for a cold scan of ten windows.

7.2 Pressing "Start stream"

1. `POST /demo/start` creates a `DemoRunner` thread holding four `StreamRunner`s, one per model, warm-started from the task-1 checkpoints.
2. For each window: every model predicts first (that prediction is logged), then the labels are revealed and fed to ADWIN as one value per flow.
3. If ADWIN reports an increase and the refractory period has passed, the model adapts on the recent windows using its own strategy (replay + EWC for ours, plain fine-tuning for the naive baseline), and the cached scores are invalidated.
4. Windows, metrics and drift events are written to the database as rows.
5. The browser polls `/stream/windows`, `/metrics` and `/drift-status` and updates the charts, the feed and the stream status row.

7.3 Flicking the "Compare models" switch

1. `main.js` stores the new value and notifies every loaded tab.
2. Each tab re-renders; `shownModels()` returns either the three story models or every ablation (minus the topology variant, which is a documented trade-off rather than a competitor).
3. Charts redraw with the same colour per model, so a reader tracking the blue line keeps tracking it.

8 • Measured results

Every figure here is in `results/` and visible in the console.

8.1 Attacks never seen in training (CSE-CIC-IDS2018, leave-one-attack-out)

Held-out attack	GNN, seed 42	GNN, seed 43	FFNN (per-flow)	XGBoost
DoS	98.6 %	98.6 %	0.1 %	90.1 %
BruteForce	99.7 %	0.0 %	0 %	0 %
DDoS	99.4 %	not run	0 %	0 %
Botnet	8.5 %	not run	0 %	0 %
Infiltration	0 %	not run	0 %	0 %

The graph detects unseen DoS in both seeds where the per-flow model sees almost nothing. Unseen BruteForce did not replicate (99.7 % in one seed, 0.0 % in the other), so this is a real effect for some attack types and not a dependable one for all. The second seed stopped after three held-out categories.

8.2 CIC-IDS2017 — seven attack types learned in sequence

Two protocols. **Interleaved** (3 seeds) tests on windows interleaved with training windows of the same session; **temporal** (5 seeds) trains on each attack's earlier traffic and tests on its latest.

Model	Macro-F1	Retention of first attack	False-positive rate
GNN + EWC + replay (ours)	0.964 ± 0.020	100 %	0.07 %
FFNN + EWC + replay (no graph)	0.928	100 %	0.04 %
GNN + replay only	0.948	100 %	0.12 %
GNN + EWC only	0.300	0 %	0.37 %
GNN naive retraining	0.289	0 %	0.45 %
XGBoost static	0.231	100 % (never learns anything new)	0.00 %

Temporal split, 5 seeds	Macro-F1	Retention of first attack	False-positive rate
GNN + EWC + replay (ours)	0.915 ± 0.031 (95 % CI 0.888–0.935)	99.8 %	0.04 %
FFNN + EWC + replay (no graph)	0.870 ± 0.009	99.7 %	0.04 %
GNN + replay only	0.848 ± 0.089	99.4 %	0.08 %
GNN + EWC only	0.377 ± 0.109	0 %	0.04 %
GNN naive retraining	0.263 ± 0.055	0 %	0.19 %

The temporal split cost our model 0.049 macro-F1 and the FFNN 0.058. Paired over seeds, the graph is ahead by +0.036 (interleaved, CI +0.011 to +0.069) and +0.044 (temporal, CI +0.019 to +0.058); with three or five seeds these are indicative, not significant.

8.3 CSE-CIC-IDS2018 — 3 seeds

Model	Macro-F1	Per seed
GNN + EWC + replay (ours)	0.881 ± 0.038	0.855, 0.863, 0.925

Model	Macro-F1	Per seed
FFNN + EWC + replay	0.836 ± 0.029	0.850, 0.855, 0.802
GNN naive retraining	0.298 ± 0.015	forgets completely
XGBoost static	0.282	—

Our model leads in all three seeds, but by 0.004 and 0.009 in two of them; the average gap is carried by the third. Stated as suggestive, not conclusive.

8.4 The product layer

Question	Measured answer
Will analysts drown in alerts?	101,913 flow alerts → 50 incidents, 84 % of them real (2018: 105 → 90 incidents at 94 % with a budget)
Does it know when not to decide?	Abstention removes all 184 false alarms on 2017 at $\alpha = 0.05$, handing 9.8 % of flows to a human
Does it spot attacks it was never taught?	Graph model 0.950 AUROC (2018, prototype score) against 0.780 for the per-flow model
Can it adapt on few labels?	1,700 labels reach 0.937 against 0.949 with everything labelled — but only with mixed sampling; uncertainty-only collapses to 0.438
Can a bad update be caught?	The safety gate rolled back one update that would have raised false alarms from 0.002 % to 0.67 %

8.5 Results reported as negative

- **EWC alone fails** (0.300 interleaved, 0.377 temporal, zero retention). Replay is what prevents forgetting. On top of replay, EWC made no detectable difference on the interleaved split but prevented two replay-only collapses on the temporal split (+0.067, CI +0.003 to +0.145, indicative).
- **Unseen BruteForce detection did not replicate** — 99.7 % in one seed, 0.0 % in the next.
- **A label-free drift trigger misses most changes** — 2 retrains instead of 16, ending at 0.338.
- **Uncertainty-only labelling fails** — a new attack the model confidently mislabels is never queried.
- **Topology augmentation is a trade-off** — robustness to randomised sources rises from 0.427 to 0.914, but the ordinary score falls from 0.964 to 0.906 and a small class is lost in two of three seeds.
- **Our model is unstable on 2018 Infiltration** — identical runs differ by up to 0.09 macro-F1.

9 · Running and operating the system

9.1 Start and stop

```
powershell -ExecutionPolicy Bypass -File scripts\run_stack.ps1 -Open      # start + open the console
powershell -ExecutionPolicy Bypass -File scripts\run_stack.ps1 -Stop    # stop everything
```

Service	Port	Process	Notes
Dashboard	8080	scripts/dashboard_server.py	serves files, proxies /api, sends security headers
Public API	8000	uvicorn src.api.app:app	results, database, forwards model calls
Model service	8001	uvicorn src.api.ml_app:app	loads four checkpoints onto the GPU

All three bind to `127.0.0.1`, so nothing is exposed to the network by default. Logs are written to `logs/stack_*.log`.

9.2 Environment variables

Variable	Effect
<code>GNNIDS_API_KEY</code>	if set, every route except /health requires this key
<code>ML_SERVICE_URL</code>	where the public API forwards model calls; unset means run the model in-process
<code>DATABASE_URL</code>	PostgreSQL/TimescaleDB instead of the local SQLite file
<code>DATASET</code> , <code>LABEL_MODE</code>	which dataset and label mode the model service serves
<code>CICFLOWMETER_JAR</code>	path to CICFlowMeter for the pcap script

9.3 Reproducing the results

```
python -m experiments.reproduce_all --dataset cicids2017
```

Runs data preparation, validation tuning, the EWC sweep, the task sequence over three seeds, the drift stream, leave-one-attack-out, the IP-remap test, the figures, the report and the database seed, in that order. Add `--dev` to any experiment for a fast partial run that writes to `results/dev/` and is never reported.

9.4 Feeding it your own traffic

```
python scripts\pcap_to_flows.py capture.pcap --api http://localhost:8000
```

Converts a capture with CICFlowMeter (installed separately), posts the flows and prints the verdicts. With an existing flow CSV, skip the conversion with `--csv flows.csv`.

9.5 Tests and continuous integration

74 tests (`python -m pytest`) covering preprocessing, graph building, the learners, the replay buffer, drift detection, the product layer, exports and all API routes, using small synthetic fixtures so they need no dataset. GitHub Actions runs them on every push.

9.6 Security posture

- Checkpoints are read with PyTorch's restricted loader; the permissive one is used only as a fallback for files inside the project's own directories.
- Optional API key; security headers on every response; a content-security policy on the console.
- Request bodies are size-limited and every value rendered in the browser is escaped.
- Proposed firewall rules are never executed. Approval records a decision, nothing more.
- Not solved on purpose, and written down as such: TLS, user accounts and roles, rate limiting, and encryption of the stored flow records.

10 · Glossary

Term	Meaning
Flow	One network conversation between two computers, summarised by about 50 numbers (duration, packet counts, byte rates, TCP flags).
Window	5,000 consecutive flows turned into one graph.
Task	A period in which a new attack type appears. The model learns tasks one after another.
Macro-F1	Detection quality averaged over every category, so a rare attack counts as much as a common one. 1.0 is perfect.
Retention	Share of the first attack type still detected after learning all the others. 1.0 means nothing was forgotten.
False-positive rate	Share of normal traffic wrongly flagged. Every 0.1 % is an alarm a person must dismiss.
BWT (backward transfer)	How much earlier tasks degraded after later training. Closer to zero is better; negative means forgetting.
EWC	Elastic Weight Consolidation: a penalty that protects the weights that mattered for earlier attacks.
Replay	Keeping a small sample of old traffic graphs and rehearsing them while learning something new.
ADWIN	A drift detector that watches the error rate and reports when its statistics shift.
Drift	Traffic changing over time, so a model trained on the past stops fitting the present.
Incident	A group of alerts that belong together (same attacker or same victim), reviewed as one item.
Abstention	The model declining to decide when it is not confident enough, handing the flow to a person.
Conformal prediction	A calibration method that turns scores into sets with a stated error rate per class.
Open-set detection	Spotting that something belongs to no known class — an attack type never trained on.
AUROC	How well two groups are separated by a score. 0.5 is a coin flip, 1.0 is perfect.
Dry run	A proposed action that is displayed and recorded but never executed.
CEF	Common Event Format: the line format most SIEM products ingest without a custom parser.
E-GraphSAGE	The graph neural network this model is built on, extended to carry features on edges.

Documentation generated from the source tree on 5 October 2026. Repository:
github.com/samarthdivekar/Continuous-learning-IDS